

Hekat TUI Roadmap - Executive Summary

Based on: Charm TUI Ecosystem Research (95KB analysis) **Research Document:**

`docs/research/TUI-CHARM-RESEARCH.md` **Date:** 2025-10-20 ---

Key Decision: Use Textual (Python)

Recommendation: Build Hekat TUI using **Textual** framework with **Rich** for enhanced output

Why Textual? -  Native Python (matches Hekat's language) -  Mature and actively maintained (2024+) -  Web-inspired API (CSS, reactive programming) -  Comprehensive widget library -  Async-first architecture -  Can run in terminal AND browser -  16.7M colors, mouse support, smooth animations -  Excellent documentation and examples

Inspiration: Charm/Bubble Tea ecosystem (Go) - the gold standard for TUI applications ---

Hekat TUI Vision

Phase 5: Interactive DSL Builder (3 weeks)

1. Wizard-Style Workflow Creator  Priority

Hekat DSL Builder [Step 2 of 4]	
Select agents for your workflow:	
☒ /deep	Deep research
☒ /ctx7	Library docs
☐ /api-architect	API design
[< Back]	[Next: Operators >]

Steps: 1. Workflow metadata (name, description, timeout) 2. Agent selection (multi-select with search) 3. Operator configuration (->, ||, +, :) 4. Review & generate DSL **2. Autocomplete DSL Editor**

```
DSL Editor - workflow.hekat

1 workflow data-pipeline:
2   /deep || /ctx7 ->
3   agent-
4
5   [ agent-1 ] ← autocomplete
6   [ agent-2 ]
7
✓ Valid DSL • 2 agents [Ctrl+R Run]
```

Features: - Syntax highlighting (DSL keywords, operators) - Real-time validation - Autocomplete suggestions - Error highlighting with locations - Line numbers - Status bar (parse status, token count) **3. Visual Workflow Builder** (Optional - Phase 7+) - Drag-and-drop agents and operators - Canvas-based workflow design - Export to DSL ---

Phase 6: Execution Dashboard (2 weeks)

Real-Time Execution Monitor ★ Critical

```
| Hekat Execution Dashboard [Ctrl+C Stop] |
|-----|
| Workflow: data-pipeline |
| Status: Running • 2/4 agents complete |
| Time: 2m 15s • Tokens: 45,000 / 100,000 |
|-----|
| ┌ Parallel Stream 1 ────────────┐ |
| | ✓ /deep [██████████] Done 2m 10s | |
| | ☞ /ctx7 [██████████░░░░░░░░] 75% 1m 05s | |
| └────────────────────────────────┘ |
|-----|
| ┌ Parallel Stream 2 ────────────┐ |
| | ⌚ /api-architect [░░░░░░░░░░░░] Waiting | |
| └────────────────────────────────┘ |
|-----|
| ┌ Live Logs ────────────┐ |
| | [14:23:15] /deep: Fetching research... | |
| | [14:23:18] /ctx7: Loading library docs... | |
| | [14:24:05] /deep: ✓ Research complete | |
| | [14:24:10] /ctx7: Processing 450 files... | |
| └────────────────────────────────┘ |
|-----|
| [Pause] [Resume] [Stop] [View Full Logs >] |
|-----|
```

Features: - Real-time progress tracking - Parallel stream visualization - Live log streaming - Token counting - Time estimates - Pause/resume/stop controls - Resource usage monitoring - Error highlighting - Log filtering (by level, agent) **Benefits:** - Visual feedback during long workflows - Easy to spot bottlenecks - Quick error identification - Professional UX ---

Phase 7: Advanced Features (2 weeks)

1. Step-Through Debugger

```
Debugger - Paused at agent-2
```

```
Variables:  
  input: {...}  
  output: null
```

```
Call Stack:  
  > agent-2 (current)  
    agent-1
```

```
[Step Over] [Step Into] [Continue]
```

2. Command Palette (Ctrl+P) - Fuzzy search for all commands - Recent commands - Keyboard shortcuts - Quick actions **3. Documentation Browser** - Integrated help - Agent reference - DSL syntax guide - Examples with copy **4. Agent Library Explorer** - Browse available agents - Filter by category - Preview agent code - Install new agents ---

Implementation Roadmap

Phase 5: Interactive DSL Builder (3 weeks)

Week 1: Wizard-Style Builder - Multi-step form implementation - Agent selection with search - Operator configuration - DSL generation **Week 2: Autocomplete Editor** - TextArea with syntax highlighting - Real-time validation - Autocomplete engine - Error display **Week 3: Polish & Integration** - Keyboard shortcuts - Help system - Testing - Documentation **Deliverables:** -  `hekat build` - Launch wizard -  `hekat edit` - Launch editor -  Textual app with 3-4 screens -  80%+ test coverage ---

Phase 6: Execution Dashboard (2 weeks)

Week 1: Core Dashboard - Live progress tracking - Parallel stream visualization - Log streaming - Status indicators **Week 2: Controls & Polish** - Pause/resume/stop - Log filtering -

Token/time tracking - Error handling **Deliverables:** -  `hekat run --watch` - Launch dashboard -  Real-time monitoring -  Professional UX -  Comprehensive logging ---

Phase 7: Advanced Features (2 weeks)

Week 1: Debugger - Breakpoint support - Step through execution - Variable inspection - Call stack view **Week 2: Command Palette & Docs** - Command palette (Ctrl+P) - Documentation browser - Agent explorer - Examples library **Deliverables:** -  `hekat debug` - Launch debugger -  Command palette in all modes -  Integrated help -  Agent discovery ---

Technical Stack

Core Libraries

Textual (v0.84+)

```
pip install textual[dev]
```

Rich (v13+)

```
pip install rich
```

Additional:

```
pip install textual-dev # Dev tools
pip install pytest-textual # Testing
```

Project Structure

```
hekat/
├─ tui/
│   ├─ __init__.py
│   ├─ app.py           # Main Textual app
│   └─ screens/
│       ├─ wizard.py    # Multi-step wizard
│       ├─ editor.py    # DSL editor
│       ├─ dashboard.py # Execution monitor
│       ├─ debugger.py  # Debugger
│       └─ help.py      # Documentation
│   └─ widgets/
│       ├─ dsl_editor.py # Custom TextArea
│       ├─ agent_list.py # Agent selector
│       ├─ log_viewer.py # Live logs
│       └─ progress.py   # Progress indicators
│   └─ styles.tcss      # Textual CSS
├─ cli.py              # Click commands
└─ compiler/           # Existing
```

CLI Commands

```
# Current (Click-based)
hekat --version
hekat info
hekat validate <file>
hekat compile <file>

# Phase 5: Interactive
hekat build          # Launch wizard
hekat edit <file>    # Launch editor

# Phase 6: Monitoring
hekat run <file> --watch # Launch dashboard
hekat run <file>        # Headless execution

# Phase 7: Advanced
hekat debug <file>     # Launch debugger
hekat docs             # Documentation browser
hekat agents           # Agent explorer
```

Design Inspiration

Color Scheme (Dark Theme)

```
# Based on Charm's aesthetic
PRIMARY = "#7D56F4"      # Purple (brand)
SUCCESS = "#73F59F"      # Green
WARNING = "#F5E373"      # Yellow
ERROR = "#F57373"        # Red
INFO = "#73C5F5"         # Blue

BACKGROUND = "#1A1B26"  # Dark background
SURFACE = "#24283B"     # Panels
TEXT = "#C0CAF5"        # Primary text
TEXT_DIM = "#565F89"    # Secondary text
BORDER = "#414868"      # Borders
```

Typography

```
/* Textual CSS */
Screen {
    background: #1A1B26;
}

.header {
    background: #7D56F4;
    color: #FFFFFF;
    text-style: bold;
}

.success {
    color: #73F59F;
}

.error {
    color: #F57373;
}

.code {
    background: #24283B;
    color: #C0CAF5;
    text-style: italic;
}
```

Key Advantages

1. Professional UX - Polished, modern interface - Smooth animations - Responsive design - Consistent with Charm aesthetic **2. Reduced Learning Curve** - Wizard guides new users - Autocomplete assists coding - Visual feedback during execution - Integrated help and documentation **3. Debugging & Monitoring** - Real-time execution visibility - Easy error identification - Performance monitoring - Step-through debugging **4. Accessibility** - Keyboard-only navigation - Screen reader support (Textual) - High contrast themes - Clear error

messages **5. Python Native** - No Go dependency - Integrates with existing Hekat code - Easy to maintain and extend - Community support ---

Success Metrics

Phase 5 Goals

-  Wizard creates valid DSL -  Editor validates in real-time -  90%+ user satisfaction -  < 1s response time

Phase 6 Goals

-  Dashboard updates < 100ms -  Handles 10+ parallel agents -  Log streaming < 10ms latency -  Professional appearance

Phase 7 Goals

-  Debugger inspects all states -  Command palette < 50ms -  Comprehensive documentation -  Full keyboard accessibility ---

Next Steps

1. **Review Research Document** (95KB) - Read: `docs/research/TUI-CHARM-RESEARCH.md` - Study code examples - Watch Charm demo videos 2. **Complete Phase 1-4** (Current) - Finish compiler (Parser, Type Checker, DAG) - Implement runtime execution - Complete voice interface - Build MCP server 3. **Phase 5 Preparation** (Week 13) - Install Textual and Rich - Prototype wizard screen - Test DSL editor component - Design app architecture 4. **Phase 5 Implementation** (Weeks 14-16) - Build wizard - Build editor - Polish and test - User testing 5. **Phase 6 Implementation** (Weeks 17-18) - Build dashboard - Integrate with runtime - Add controls - Performance tuning ---

Conclusion

The Charm TUI ecosystem research provides a **clear roadmap** for building a **professional, polished** Hekat CLI experience. By using **Textual (Python)**, we can achieve **95% of Charm's aesthetic** while staying in Python and integrating seamlessly with Hekat's existing codebase.

Key Takeaways: 1. **Textual is the right choice** for Hekat TUI 2. **Wizard-style builder** reduces learning curve 3. **Real-time dashboard** provides visibility 4. **Professional UX** matches modern CLI tools 5. **Phased approach** ensures quality **Timeline:** 7 weeks (Phases 5-7)

Impact: Transforms Hekat from CLI to professional TUI application **Inspiration:**

Charm/Bubble Tea ecosystem --- **Resources:** - Research Document: `docs/research/TUI-`

`CHARM-RESEARCH.md` (95KB) - Textual Docs: <https://textual.textualize.io/> - Rich Docs:

<https://rich.readthedocs.io/> - Charm Examples: <https://github.com/charmbracelet> **Hekat:**

Ancient wisdom meets modern agent coordination. *With a beautiful TUI to match.*